Edge AI Surveillance Systems Using TinyML on ESP32 Microcontrollers with CNN Quantization for Privacy-Preserving Local Processing: A Literature Review

A Review for Research Publication

Abstract

Evidence on edge AI surveillance systems using TinyML on ESP32 microcontrollers with CNN quantization for privacy-preserving local processing remains limited. While studies demonstrate feasibility on comparable low-power microcontrollers like MAX78000 (up to 180 fps, 196 µJ per inference) [8], there is a distinct gap in deploying these highly optimized pipelines on ubiquitous hardware like the ESP32. Privacy-preserving local processing is consistently supported through edge-based anonymization, semantic scene graph generation, and chaotic frame scrambling [11, 5, 4]. Lightweight CNN optimizations, including depthwise separable convolutions and adaptive 4-bit quantization, enable real-time object detection on resource-constrained devices, with strong emphasis on latency and bandwidth reduction via edge-cloud hybrids [12, 3]. This review synthesizes advancements in TinyML for surveillance amid rising smart city demands, establishing the foundational literature that justifies the localized, MQTT-driven architecture of the EagleEye Smart Intrusion System. Future work must bridge the gap between theoretical micro-architectures and practical ESP32 deployments to realize the full potential of localized privacy preservation.

Keywords: edge computing, surveillance, privacy preserving, latency, bandwidth reduction, TinyML, ESP32, quantization

1 Introduction

The proliferation of video surveillance in smart cities, campuses, and industrial sites has transformed monitoring into a cornerstone of public safety. Traditional centralized systems, reliant on continuous cloud processing of raw video streams, face critical bottlenecks: high latency from data transmission, massive bandwidth demands, and privacy risks from exposing personally identifiable information in transit [9, 11]. Edge AI, particularly Tiny Machine Learning (TinyML) on ultra-low-power microcontrollers, emerges as a paradigm shift by enabling local inference with convolutional neural networks (CNNs) quantized to fit severe memory constraints (e.g., <0.5 MB). This approach inherently preserves privacy through on-device processing without raw data upload [8].

Despite these promises, integrating TinyML surveillance on highly popular platforms like the ESP32 microcontroller—known for its integrated Wi-Fi and ∼520 KB SRAM—remains underexplored. Most empirical work targets analogous hardware like STM32 or

MAX78000. Challenges persist in balancing accuracy, inference speed, energy efficiency, and privacy for tasks like object detection amid heterogeneous edge environments. This review synthesizes evidence on edge AI surveillance systems using TinyML with CNN quantization, focusing on privacy-preserving local processing. It clarifies feasibility, optimizations, and gaps relative to deploying these systems on constrained hardware, providing the academic foundation for practical systems like EagleEye that leverage MQTT, Python gateways, and custom TinyML pipelines.

2 Methods

2.1 Search Strategy and Study Selection

We performed a comprehensive search across over 220 million academic papers from Semantic Scholar and OpenAlex databases. The search strategy employed hybrid semantic and keyword-based retrieval targeting: *edge-computing, surveillance, IoT, TinyML, microcontroller, CNN-quantization, and privacy-preserving.* Initial searches identified 280 records. After duplicate removal and relevance-based filtering, 67 records were screened against eligibility criteria focusing on edge-AI deployment, surveillance tasks, and hardware evaluations. Ultimately, 20 papers published from 2018 onward were included in the final qualitative synthesis.

3 Results and Thematic Findings

3.1 Privacy-Preserving Local Processing in Edge Surveillance

Edge-based anonymization and semantic processing consistently enable privacy protection by avoiding continuous raw video transmission. Approaches include minor/face detection (92.1% accuracy [11]), in-memory encryption, and scene graph generation to safeguard human images [9, 5]. Chaotic scrambling methods provide lightweight end-to-end privacy at the edge, computationally feasible versus heavy cryptographic alternatives [4]. By acting as a "privacy buffer," edge devices ensure that sensor payloads (like an MQTT trigger) are only uploaded upon verified detection, vastly outperforming centralized models in data ownership [9].

3.2 CNN Quantization and Lightweight Optimizations

Quantization techniques dominate model compression for TinyML. 8-bit quantization-aware training achieves <0.5 MB memory footprint (e.g., 422k parameters [8]), while adaptive 4-bit INT4 techniques adjust scaling/shifting for biased activations, sometimes surpassing full-precision baselines [3]. Depthwise separable convolutions significantly reduce complexity, allowing lightweight models to achieve 16 fps with 89% precision [12]. Architectures like MobileNet or custom shallow networks paired with grayscale input down-sampling (48×48) are critical to overcoming the constraints of microcontrollers.

3.3 Microcontroller Deployment and Constrained Hardware

Deployments on specialized low-power MCUs like the MAX78000 achieve 180 fps and 196 µJ/inference [8]. Arduino Portenta and STM32 arrays show strong feasibility for on-device training and quantized inference [1, 8]. However, a prominent gap exists regarding direct metrics for ESP32 deployments. While general edge devices hit real-time metrics (16-36.7 fps [12, 10]), the ESP32's unique balance of dual-core processing and integrated networking requires nuanced optimization, such as separating the I2S camera loop from the inference payload.

3.4 Surveillance Tasks and Edge-Cloud Hybrids

Object tracking (e.g., pedestrians, secure-entry monitoring) dominates the literature, with edge-cloud cooperation boosting precision via secondary verification [10, 7]. This hybrid approach perfectly mirrors modern IoT implementations where the ESP32 handles preliminary TinyML detection entirely offline, communicating lightweight event pulses via MQTT to a secure Python edge-gateway, which then interfaces with heavy cloud storage (e.g., Firebase) to alert frameworks like React Native.

Table 1: Summary of Evidence: Key Literature Findings

Theme	**Key Finding**	**Population Applicability**	**Effect Direction**	**Confidence Level**	**Supporting Studies**
Privacy-Preserving Local Processing	92.1% minor classification accuracy; chaotic scrambling feasible	Smart city/campus surveillance	Positive	Moderate (consistent findings with reasonable design quality)	Myneni et al. [9], Yuan et al. [11], Huang et al. [5]
CNN Quantization and Lightweight Optimizations	180 fps, 196 µJ/inference; 0.91% mAP gain; 4-bit > state-of-art	TinyML edge inference	Positive	Strong (consistent across multiple studies with metrics)	Moosmann et al. [8], Chin et al. [3], Kim et al. [6]
Microcontroller Deployment and Performance	16-180 fps on MCUs; gap identified for ESP32	Low-power MCUs (no ESP32)	Positive	Limited (sparse MCU-specific metrics)	Zhao et al. [12], Moosmann et al. [8]
Surveillance Tasks and Edge-Cloud Hybrids	76.7-80.7% AP pedestrians; 92-93% precision counting	Industrial/urban surveillance	Positive	Moderate (consistent real-world tasks)	Xu et al. [10], Lyu et al. [7]

4 Discussion and Future Directions

Quantization emerges as the cornerstone enabling TinyML surveillance. 8-bit [8] and adaptive 4-bit methods [3] preserve accuracy under memory extremes (<0.5 MB) by dynamically mitigating biased activation losses inherent to standard commercial CNNs. This explains the superiority of constrained models over unoptimized baselines. Depth-wise separable convolutions synergize with this quantization, slashing parameters pre-compression [12, 10].

Privacy gains are derived mechanistically from local feature extraction. By blocking identifiable data leaks without transmission, edge processing acts as a direct counter to cloud vulnerabilities [11, 4]. The confidence is high regarding quantization efficacy and moderate for privacy architectures.

The structural literature reveals a critical gap: **there are near-zero comprehensive ESP32 deployments explicitly benchmarked for surveillance tasks.** The ESP32's dual-core architecture, widespread cost-effectiveness, and built-in Wi-Fi make it arguably the most important target for "democratized" smart security, yet the literature predominantly focuses on non-networked MCUs (like the MAX78000) or heavy generic edge devices.

Future practical systems must address this gap by validating quantized CNNs (Int8) specifically on ESP32 hardware for human detection. Integrating these sensors with robust, low-bandwidth protocols like MQTT, utilizing local Python gateways to abstract load, and rendering results in real-time cross-platform ecosystems (like React Native) represents the exact intersection of privacy-by-design and modern full-stack surveillance.

5 Conclusion

Edge AI surveillance via TinyML with CNN quantization supports privacy-preserving local processing on low-power microcontrollers, achieving incredibly low energy profiles and reliable real-time tasking [8, 12]. Edge outperforms continuous-capture cloud systems in latency, bandwidth, and privacy via local feature extraction [10, 4]. The pivotal uncertainty remaining in academic literature is specific viability on the ESP32 platform, as its memory map and Wi-Fi load uniquely alter quantization trade-offs unseen in offline reviewed platforms. Consequently, projects utilizing hardware like the ESP32-CAM to fill this precise performance gap provide highly valuable empirical data to the edge-AI community, transforming edge analytics from conceptual prototype to scalable, secure production.

References

[1] Aatab, S., & Freitag, F. (2023). Towards a Library of Deep Neural Networks for Experimenting with on-Device Training on Microcontrollers. *2023 IEEE 9th World Forum on Internet of Things (WF-IoT)*. IEEE.

[2] Banbury, C. R., et al. (2024). Wake Vision: A Tailored Dataset and Benchmark Suite for TinyML Computer Vision Applications.

[3] Chin, H.-H., Tsay, R.-S., & Wu, H.-I. (2022). An Adaptive High-Performance Quantization Approach for Resource-Constrained CNN Inference. *2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. IEEE.

[4] Fitwi, A., Chen, Y., & Zhu, S. (2021). Lightweight Frame Scrambling Mechanisms for End-to-End Privacy in Edge Smart Surveillance. IEEE.

[5] Huang, A. Y. C., et al. (2023). Semantic Privacy-Preserving for Video Surveillance Services on the Edge. *Proceedings of the Eighth ACM/IEEE Symposium on Edge Computing*.

[6] Kim, S., & Kim, H. (2020). Mixture of Deterministic and Stochastic Quantization Schemes for Lightweight CNN. *2020 International SoC Design Conference (ISOCC)*. IEEE.

[7] Lyu, Z., & Luo, J. (2022). A Surveillance Video Real-Time Object Detection System Based on Edge-Cloud Cooperation in Airport Apron. *Applied Sciences*.

[8] Moosmann, J., et al. (2023). TinyissimoYOLO: A Quantized, Low-Memory Footprint, TinyML Object Detection Network for Low Power Microcontrollers. *2023 IEEE 5th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*.

[9] Myneni, S., et al. (2022). SCVS: On AI and Edge Clouds Enabled Privacy-preserved Smart-city Video Surveillance Services. *ACM Transactions on Internet of Things*.

[10] Xu, Z., Li, J., & Zhang, M. (2021). A Surveillance Video Real-Time Analysis System Based on Edge-Cloud and FL-YOLO Cooperation in Coal Mine. *IEEE Access*.

[11] Yuan, M., et al. (2020). Minor Privacy Protection Through Real-time Video Processing at the Edge. *2020 29th International Conference on Computer Communications and Networks (ICCCN)*.

[12] Zhao, Y., Yin, Y., & Gui, G. (2020). Lightweight Deep Learning Based Intelligent Edge Surveillance Techniques. *IEEE Transactions on Cognitive Communications and Networking*.